

Document number: P2957R0
Date: 2023-08-15
Audience: SG21
Reply-to: Andrzej Krzemiński <akrzemi1 at gmail dot com>
Iain Sandoe <iain at sandoe dot co dot uk>

Contracts and coroutines

This paper proposes how preconditions, postconditions and assertions (contract annotations) should interact with coroutines.

Coroutines generalise regular functions in a specific manner: control can return to the caller even though the function body has not finished executing. This gives rise to some doubts as to whether the semantics of contract annotations as defined for regular functions still apply.

Assertions

In-body assertions are possibly the easiest one to handle as they do not involve a contract between two parties. They are an implementation detail. An assertion is expected to hold when a corresponding statement of the coroutine body is executed. We know when macro `assert()` is evaluated in a coroutine, and we expect a new assertion from the contract support framework to behave analogously.

Coroutine properties

A regular function *definition* becomes a coroutine when one of the 'co_' keywords is encountered in its body. Thus one cannot tell if a function is a coroutine from either its declaration or invocations, for example:

```
awaitable<int> session(int id); // may be a coroutine, may be a function

awaitable<int> default_session()
{
    awaitable<int> s = session(0); // maybe invoking a coroutine, maybe a function
    return s;
}
```

In this example, since we have the function body, we can see that `default_session` is a regular function even though it returns an *Awaitable*. We know less about `session`: it could also be a factory function, like `default_session`, or it could be a coroutine.

When we invoke a coroutine, from the caller's perspective it behaves as a factory function for creating a *coroutine return object* (current implementations call this factory function a "*ramp function*"). Such a *ramp function* can surely have a contract. For instance, `session` could require that the passed `id` has a non-negative value.

An important difference between regular functions and coroutines, is that when the callee returns, for a regular function all automatic state of the function is complete; conversely, for a coroutine that has suspended, some or all of that state (and parameter values) might have been captured in preparation for a potential resumption. Similarly, observable side-effects of a coroutine can (most likely will) change at some arbitrary point after the dialogue with the caller has concluded. Thus, when the *ramp function* (the call to `session(id)` above) returns, and has produced a *coroutine return object*, we might have expectation of some sub-set of state of this return object but, in general, that could be more limited than the expectations applicable to a regular function.

Contract annotations and function contract

A *function contract* is all the information the caller needs to know to use the function correctly. It is a very wide notion and includes things like the number, type and the interpretation of the arguments, runtime complexity. Generic terms "precondition" and "postcondition" are also very broad and indicate, respectively, all that function expects when it is called, and all that it guarantees upon successful return when its expectations are satisfied. In contrast, contract annotation `pre` (for precondition) and contract annotation `post` are a tiny subset of the above, and indicate a very specific thing: a predicate (a C++ expression, ideally a pure one, returning `bool`) that is expected to evaluate to `true` if called at appropriate place. "Expected" means that if it should not evaluate to `true` we can be sure we have a bug somewhere. The "appropriate place" for non-coroutine functions is:

1. For preconditions: when a function is invoked, immediately after function parameters have been initialized.
2. For postconditions: when a function returns normally, after the returned value has been initialized, all automatic objects in the function body have been destroyed, before function parameters have been destroyed.

The runtime-checking of a precondition ensures that unexpected combinations of function arguments (and the program state) are never used in the function body. The runtime-checking of a postcondition ensures that a caller obtains an object (or other parts of the program state) in a desired state that will satisfy the preconditions of the next functions that the caller may call:

```
awaitable<int> cancelable_session(int id)
    [[post r: is_cancelable(r)]];

template <typename T>
void manage(awaitable<T> session)
    [[pre: is_cancelable(session)]];

void test()
{
    awaitable<int> session = cancelable_session(1);
    // I need guarantee that is_cancelable(session) is true
    // to make sure that I am calling manage in contract
    manage(session);
}
```

Thus, preconditions and postconditions are relations between a caller and the callee. The caller invokes the callee. The caller is only aware of the callee signature: not its body (or implementation). A caller does not and cannot know whether it is calling a coroutine or a regular function.

Preconditions on coroutines

Given the above, the semantics of a precondition become quite obvious. It is a predicate that is expected to hold when the caller invokes the function overload corresponding to the coroutine, immediately after the function parameters have been initialized, but before any of the coroutine actions, potentially consuming the parameters, are taken. These actions include:

1. Possible allocation of a coroutine frame [\[dcl.fct.def.coroutine\]/9](#) which might have a user-supplied allocator taking the parameters as arguments.
2. Copying of the parameters to the coroutine frame and subsequent initialization of the coroutine promise object (which may take those copies as arguments to its constructor) [\[dcl.fct.def.coroutine\]/5.7](#).

[\[dcl.fct.def.coroutine\]](#) specifies that a coroutine behaves as if its *function-body* were replaced by:

```
{
    promise-type promise promise-constructor-arguments;
    try {
        co_await promise.initial_suspend();
        function-body
    } catch ( ... ) {
        if (!initial-await-resume-called)
            throw ;
    }
}
```

```

    promise.unhandled_exception();
}
final_suspend:
    co_await promise.final_suspend();
}

```

So, at present, we have no mechanism to identify the phase "after the call but before any of the coroutine transformations are applied". Further, we want to give implementations the freedom to evaluate the preconditions either in the caller or in the callee, and in either case the program should behave the same. This means that we need to identify the precondition actions as distinct from the function body (in the case that the callee implements them) otherwise the model above would cause them to act after the initial suspend.

The following shows a motivating use case for preconditions on coroutines.

```

generator<int> sequence(int from, int to)
    [[pre: from <= to]];

```

Postconditions on coroutines

Postconditions on coroutines are trickier. This is because, unlike in regular functions, the returning to the caller and ending the coroutine happen potentially at two different points in time: the control is returned to the caller, but the coroutine may be suspended, and resumed, via the coroutine handler much later (and therefore completely unrelated to the original caller/callee pair) .

In normal functions, a postcondition often talks about a return value, which is created as the last thing in the function body. It is natural to expect that the postcondition is established at the end. "What happens at the end of the function" (1) and "what happens when the control returns to the caller" (2) are the same thing. However, when we need to distinguish between them, in the case of coroutines, only the latter involves the caller.

Before we propose the desired semantics, we need to highlight two things: one is a property of asynchronous computations in general, the other is specific to C++ coroutines.

In an asynchronous call, the caller, by definition, is not interested in the results of the asynchronous operation *A*. The caller will not obtain the return value (if any) from *A*. The caller will not be informed if *A* succeeded or failed. The caller only fires *A* and forgets about it. *A* only communicates its results via some global state, or callbacks. so, whatever happens at the end of *A* is of no interest to the caller. Contrast this with the properties of the postcondition: it is a guarantee given *to the caller*, so that the caller can satisfy the preconditions of subsequent operations that it calls *sequentially*.

However, every asynchronous operation consists of a small initial sequential part, like putting a task onto an asynchronous task queue, or calling the *ramp function* of the coroutine (creating the coroutine frame, storing a handle to it, evaluating the coroutine body until the first suspension, and building the return object). The caller may want to know if this operation succeeded. "did I manage to launch the asynchronous operation?" This is a guarantee for the caller that qualifies for a postcondition: it may affect the decisions of the caller.

In the following example, the caller starts a coroutine, only passes its handle to someone else, but is not interested in how the coroutine ended, and, in fact, the caller ends before the coroutine is resumed. Yet, the caller is interested in how the synchronous portion of the coroutine (the ramp function) ended:

```

awaitable<int> cancelable_session(int id)
    [[post r: is_cancelable(r)]];

void caller()
{
    awaitable<int> s = cancelable_session(1);
    [[assert: is_cancelable(s)]];
    global_cancelable_sessions.push(std::move(s));
}

```

We might be cautious here, since "the sequential part" of a coroutine can also depend on the conditions that are found when each possible suspension point is evaluated; that is, a coroutine could validly complete completely synchronously. This argues for limiting post conditions to state represented by the return object (and reiterating that neither internal state nor observable side-effects can be considered to be in any way known).

But don't we want to know what happened at the end of the coroutine? The answer is "possibly", but that would be a different kind of contract because there is not necessarily a single "end" and also such conditions do not represent an agreement between the caller and callee. The caller is certainly not able to guarantee to verify them with runtime checks at any point in time. So a hypothetical guarantee concerning the state of the coroutine at some arbitrary 'end point' (perhaps a suspension point) is something that would have to be checked inside a coroutine in the manner of assertions.

A second point, already mentioned, is that a postcondition is something related to the caller, and the only thing that the caller sees is the function declaration and the invocation of the function. The caller does not see the function body, so it has no way of knowing if it is invoking a coroutine or a regular function. The implementation should be able to verify the declared postcondition by invoking the predicate in the caller immediately after the ramp function finishes:

```
awaitable<int> cancelable_session(int id)
[[post r: is_cancelable(r)]];

void caller()
{
    awaitable<int> s = cancelable_session(1);
    // should be able to verify the postcondition here
    // ...
}
```

Therefore, we propose that postconditions on coroutines be allowed and that they should express the desired state of the program immediately after invoking the ramp function of the coroutine. More formally, the evaluation of the coroutine postcondition is sequenced immediately after the initialization of the result object of a call to a coroutine. The evaluation of the postconditions is unsequenced with respect to the destruction of the coroutine local state. Function parameter names that appear in postconditions refer to original function parameters rather than to their copies ([\[dcl.fct.def.coroutine\]/13](#)).

(Note that "the initialization of the result object of a call to a coroutine" is not the same as the call to `promise.get_return_object()`. The latter happens earlier, and is followed by the call to `initial_suspend`. In the extreme case the type of "the initialization of the result object of a call to a coroutine" can be some X, whereas `decltype(promise.get_return_object())` can be some other type Y only convertible to X, and in that case the postcondition must refer to type X because this is the only type that the caller sees.)

In other words, precondition and postcondition annotations should have the same behavior (modulo copies of arguments and returned value) as if the call to the coroutine was wrapped into a factory forwarding function with analogous contract annotations:

```
awaitable<int> f1(int i) // coroutine
[[pre: p(i)]]
[[post r: q(r)]];

awaitable<int> f2(int i); // coroutine

awaitable<int> ff2(int i)
[[pre: p(i)]]
[[post r: q(r)]];
{
    return f2(i);
}

void caller()
{
    f1(1); // these two calls have analogous
```

```
    ff2(1); // precondition and postcondition semantics
}
```

Coroutine-specific guarantees

Coroutines, due to their unique nature, may require different kinds of guarantees to be expressed. One such guarantee has already been mentioned: what happens when the coroutine is finished in a natural manner: not via an exception, not via canceling the coroutine with `.destroy()`. This, in fact could be expressed with ordinary `[[assert: _]]` at the end of the coroutine scope, assuming that it has a single exit point. Otherwise, a new declaration would have to be invented, probably not exposed to the callers in the declaration:

```
awaitable<void> seq(State& s)
{
    [[co_post: s.done()]]; // guaranteed at the end of coroutine body
    // coroutine body
}
```

We are not proposing that, or anything else from this section at this point. This is only to indicate a possible future direction, composable with regular post- and pre-conditions.

Another suggested guarantee is that for the following declaration:

```
awaitable<int> fun();
```

We want to specify that the values `co_await`d on the returned *Awaitable* satisfy a certain constraint. While conceivable, note that this guarantee is not specific to coroutines. In the above declaration, `fun` need not be a coroutine, but it may still be useful to express the same `co_await`-guarantee. Also consider this example:

```
awaitable<void> test(awaitable<int>& aa)
{
    int i = co_await aa; // how to guarantee anything?
    co_yield i + 1;
}
```

Here, we might want to have a guarantee that `aa` yields only certain values, but there is no coroutine in sight that would be making this guarantee. Such a guarantee would have to be a property of the *Awaitable*.

Even if the coroutine were in sight, enforcing such a guarantee would be difficult. When a coroutine yields a value, this value is never seen directly by the consumer. It is first passed to function `yield_value`, then there may be an arbitrary number of additional functions processing the value, depending on the implementation of the promise type and the *Awaitable*, and then `operator co_await` is invoked. Each of these functions can arbitrarily change the original value, so that the value yielded by the coroutine may be arbitrarily different from the value that the consumer obtains.

This problem is somewhat similar to wanting to express a guarantee that the callback returned from the following function always returns positive values:

```
std::function<int()> positive_values();
```

While useful, it is not implementable in the current model for contract support. In the current model, we need to have a clear single place where the implementation can runtime-check the predicate expressing the guarantee. Even if it was somehow implementable, that would be a different kind of contract: it is *not* what the call to the coroutine's ramp function guarantees to the caller at the end of the call. Instead, it is something that the coroutine guarantees to unpredictable parties (we do not know how many different actors will `co_await` on the produced *Awaitable*) at unpredictable places. Also, in the case of `std::generator`, a coroutine need not produce an *Awaitable* at all.

Implementability

Currently the only compiler to implement contracts is GCC. It implements `[[assert: _]]` as proposed in this paper.

The implementation of the preconditions differs from what we propose: the precondition is evaluated after the initial suspension point. This means that when a coroutine with an initial suspend is called and never resumed, the precondition is never evaluated. Postconditions, even though they are syntax-checked and allow referencing the coroutine return object, are never evaluated.

Feedback from the GCC team indicates that this behavior is the outcome of the simplest implementation choices (reflecting that both topics were still fluid) rather than a specific design. [P0542R5] didn't pay attention to how pre- and post conditions should work with coroutines. As a result, given that coroutines are functions, especially if we consider the callee-side checking implementation, these requirements were ambiguous for preconditions and unimplementable for postconditions. The requirements were:

A precondition is checked by evaluating its predicate immediately before starting evaluation of the function body. [...] A postcondition is checked immediately before returning control to the caller of the function. [Note: The lifetime of local variables and temporaries has ended. [...] — end note].

[P0542R5] instructed that preconditions are evaluated before the function body, whereas the wording for coroutines required that the function body is evaluated after the coroutine frame allocation, initialization of the promise and the initialization of the return object has been performed, and after the initial suspend. The literal reading of the coroutine requirement would mean that the precondition is evaluated after the initial suspend, but this is not the desired behavior. The description of the postcondition is illogical in the context of coroutines: the place "immediately before returning control to the caller" and where "the lifetime of local variables and temporaries has ended" simply does not exist.

It is believed that the semantics from this paper can be implemented at least in GCC by putting the runtime checks for pre- and post-conditions inside the ramp function . These semantics can be also emulated by the programmers by wrapping the call to a ramp function of a coroutine with a forwarding function containing the desired pre- and post-conditions, as illustrated earlier.

References

- [P0542R5] — G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup, "Support for contract based programming in C++", (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0542r5.html>).